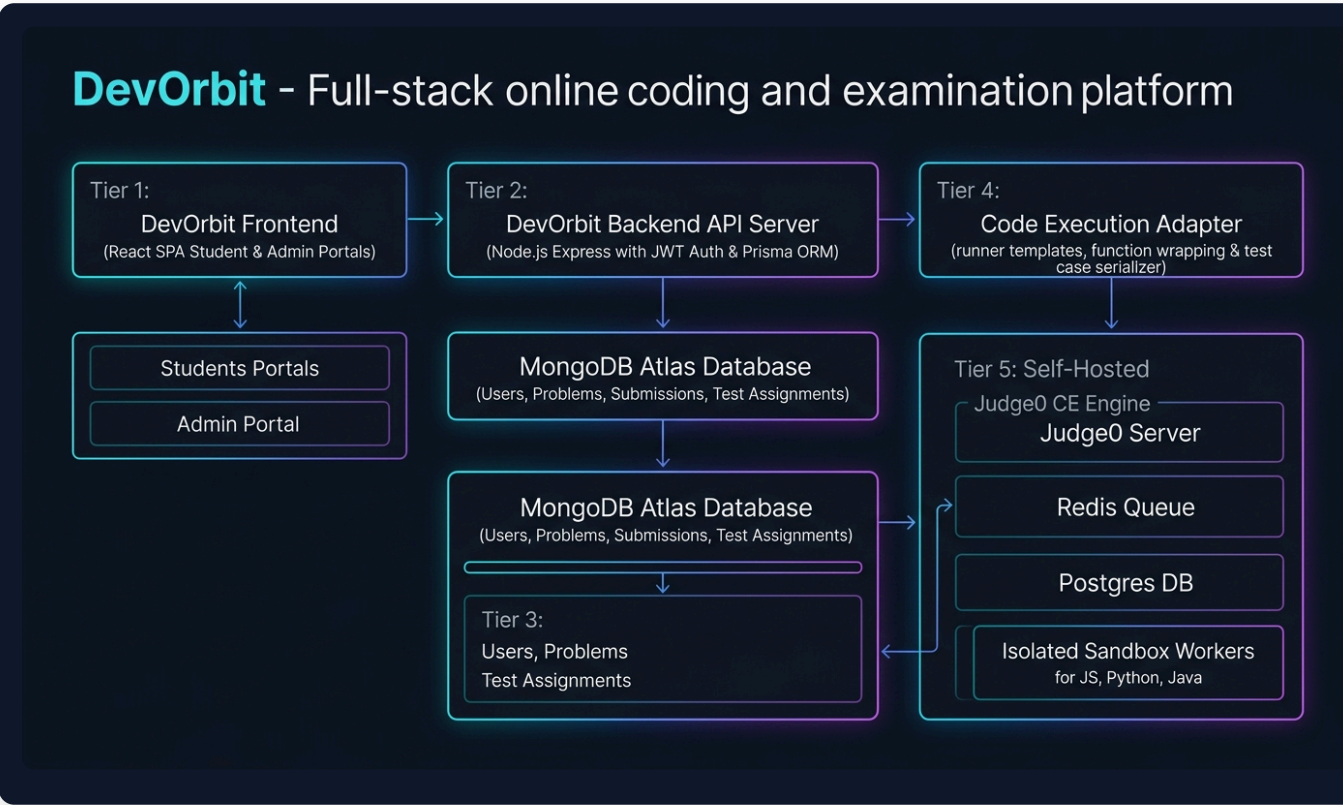


1. System Architecture Overview

DevOrbit (Hackercit) is an enterprise-grade full-stack online coding, contest, and automated assessment platform. The system follows a **decoupled, tiered architecture** designed for high throughput, strict security isolation, and real-time evaluation of untrusted student code.



2. Architectural Tier Breakdown

Tier 1: Frontend Client (hacker.jsx)

Single Page Application built with React and custom styling. Contains zero hardcoded problem content.

- **Student Portal:** Practice problems catalog, contest mode, timer, monaco editor.
- **Admin Portal:** Problem creation, MCQ management, contest controls.

Tier 2: Express Backend Server

Node.js Express REST API server running on port 4000 with JWT Authentication middleware.

- `problemController.js`: Dynamic MongoDB problem queries.
- `testController.js`: Exam lifecycle and attempt state machine.
- `submissionController.js`: Code execution dispatcher.

Tier 3: MongoDB Atlas Database

Single source of truth accessed via Prisma ORM for all persistent application data.

- User: Student/Admin profiles and credentials.
- Problem: Coding challenges and theory MCQs.
- TestAssignment & TestAttempt: Exams and live attempts.
- Submission: Recorded execution submissions.

Tier 4: Execution Harness Adapter (runner/)

In-memory adapter bridging LeetCode-style problems to code execution engines.

- `templates.js`: Single-file wrapper code generator.
- Injects `ListNode`, `TreeNode` helpers.
- Serializes test case output to JSON stdout.

3. Tier 5: Self-Hosted Judge0 CE Execution Engine

Untrusted student code is isolated and executed inside a self-hosted Judge0 CE container stack.

Component	Technology	Port	Description / Role
Judge0 Server API	Judge0 CE v1.13.1	2358	REST API handling execution requests with X-Auth-Token header protection.
Redis Queue	Redis 7-Alpine	6379	In-memory submission task queue managing concurrent execution jobs.
PostgreSQL DB	Postgres 15-Alpine	5432	Persistent submission log and system metadata store.
Isolate Workers	Linux Kernel Isolate	Internal	Sandboxed container workers executing JavaScript (63), Python (71), and Java (62).

4. Security, Isolation & Resource Governance

Sandbox Security Rules

- **Process Isolation:** Managed via Linux Kernel cgroups v2, chroot, and seccomp system call filters.
- **CPU Limit:** Enforced at 3.0 seconds per submission with a 1.0s grace period.
- **Memory Limit:** Hard capped at 256 MB (256,000 KB) per sandbox execution.
- **Process Fork Limit:** Capped at 60 max processes/threads to prevent fork bomb attacks.
- **Network Access:** ENABLE_NETWORK=false disables outbound socket creation, preventing reverse shells and network port scanning.

5. Scalability Architecture for 500 Concurrent Students

To support 500 simultaneous student submissions during live proctored examinations without socket exhaustion:

- **Horizontal Scaling:** 4 to 8 Judge0 worker containers running on an AWS EC2 compute cluster (c6i.4xlarge, 16 vCPUs, 32 GB RAM).
- **Asynchronous Task Queueing:** Submissions enqueued in Redis process concurrently across worker nodes without blocking Express Web HTTP threads.
- **Load Balancing:** AWS Application Load Balancer routing port 2358 internal traffic.

6. Final Compliance Verification Summary

Verification Item	Status	Implementation Details
Single Source of Truth	VERIFIED	MongoDB Atlas stores 100% of coding problems; 0 hardcoded arrays in frontend.
Reference Solution Privacy	SECURE	Reference solutions are backend-only and never exposed to Student APIs.
MCQ Data Separation	VERIFIED	Practice API excludes MCQs (type: coding); Exam Workspace loads assigned MCQs by ID.
End-to-End Test Suite	100% PASS	Verified via automated QA test suite on localhost environment.